

Dynamic Weighted Late Fusion Pipeline for Multi-Modal Drone Detection



Sebahattin Saral, Ammar Omerika, Ervin Dragović

International University of Sarajevo

2026

Multi Modal Fusion for Drone Detection

BY

Sebahattin Saral – Software Engineering

Ammar Omerika – Software Engineering

Ervin Dragović – Software Engineering

A graduation project submitted in partial fulfillment of the
requirements for the degree of Bachelor of Science at the
Faculty of Engineering and Natural Sciences
International University of Sarajevo

June 2026

APPROVAL PAGE

I certify that I have supervised and read this study and that in my opinion, it conforms to acceptable standards of scholarly presentation and is fully adequate, in scope and quality, as a graduation project for the degree of Bachelor of Science at the Faculty of Engineering and Natural Sciences.

.....

Dr. Özge Büyükdağlı

Mentor

.....

Dr. Kanita Karađuzović - Hadžiabdić

Mentor

.....

Dr. Migdat Hodžić

Mentor

DECLARATION

We hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. We also declare that, as required by these rules and conduct, We have fully cited and referenced all material and results that are not original to this work.

Sebahattin Saral

Signature

Ammar Omerika

Signature

Ervin Dragović

Signature

Date

INTERNATIONAL UNIVERSITY OF SARAJEVO

DECLARATION OF COPYRIGHT AND AFFIRMATION OF FAIR USE OF UNPUBLISHED WORK

Copyright © 2026 by Sebahattin Saral, Ammar Omerika and Ervin Dragović.
All rights reserved.

Multi Modal Fusion for Drone Detection

No part of this unpublished work may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without prior written permission of the copyright holder and IUS Library.

Affirmed by:

Sebahattin Saral

Signature

Ammar Omerika

Signature

Ervin Dragović

Signature

Date

ACKNOWLEDGEMENTS

[I/We wish to express my indebtedness to all those who offered their assistance during my research.

I/We would like to thank]

We should thank someone.

LIST OF ABBREVIATIONS

AI – Artificial Intelligence

API – Application programming interface

CVAT – Computer Vision Annotation Tool

EO – Electro Optical

FOV – Field of view

FPS – Frames per second

HSV – Hue Saturation Value

IOU – Intersection over Union

IR – Infrared

RGB – Red Green Blue

UAV – Unmanned Aerial Vehicle

UI – User Interface

YOLO – You only look once

TABLE OF CONTENTS

APPROVAL PAGE	4
DECLARATION.....	5
DECLARATION OF COPYRIGHT AND AFFIRMATION OF FAIR USE OF UNPUBLISHED WORK	6
ACKNOWLEDGEMENTS.....	7
LIST OF ABBREVIATIONS.....	8
ABSTRACT	11
LIST OF TABLES.....	12
LIST OF FIGURES.....	13
1. INTRODUCTION	14
1.1 Background of Study	14
1.2 Problem Statement and Research Aims	14
1.3 Research Questions and Objectives	14
2. RELATED WORK	15
3. DATASET PREPARATION	16
3.1. Naming Convention	16
3.2. Extracting Individual Frames	16
3.3. Categorizing Frames	16
4. IMAGE LABELING.....	21
5. BASELINE MODELS	22
6. FUSION.....	23
7. FUSION PIPELINE.....	25
7.1 Input data selection	25
And then for our second condition we have a target size. The target size is calculated from the long side of the predicted box. For an agreement pair, when two sensors are deciding on the target size, it's calculated from the mean of two long sides of the boxes.	26
7.2 Pipeline Stages	26
7.3 Decision Logic	29
8. RESULTS	30
9. DISCUSSION.....	31

10. CONCLUSION.....	32
REFERENCES.....	33

We should seperate the initial administrative pages so they are counter using roman numerals.

ABSTRACT

Multi Modal Fusion For Drone Detection

As they say, the abstract shall be written last.

LIST OF TABLES

Table 1 Dataset Characteristics	15
Table 2 Dataset naming convention parameters.....	15
Table 3 Dataset Statistics	18
Table 4 Brithness ranges	25

LIST OF FIGURES

Figure 1 The difference in framerate causing a missing frame	15
Figure 2 Template of the file naming convention and an example of a named file	16
Figure 3 A frame that was categorized as cluttered, but should be categorized as clear	17
Figure 4 BitFlipper UI	17
Figure 5 Decision scheme for categorizing frames as cluttered or clear	18
Figure 6 Image with a labeled drone on the CVAT annotation platform	20
Figure 7 A drone label in YOLO annotation format	20
Figure 8 YOLO detection object	21
Figure 9 Detection system that implements early fusion	22
Figure 10 System that implements late fusion.....	23
Figure 11 Abstract view of the system.....	24
Figure 12 Fusion pipeline stages.	26

1. INTRODUCTION

1.1 Background of Study

Over the previous decade, the deployment of Unmanned Aerial Vehicles (UAVs) has shown a significant increase. Following the increase in their deployment, concerns were raised on the potential threats involving the usage of UAVs for acts of terrorism, smuggling, unauthorized surveillance, and other problematic activities [1].

To combat the increased number of UAV threats, a reliable way to detect UAVs is of key importance. Many of the existing drone detection approaches use only a single modality (type of sensor data) such as Acoustic, Radio frequency, Radar or Visual Detection. However, each of these methods has their own limitations, and one approach to overcome the limitations of individual methods is fusion, in which different sensors inputs are combined through decision mechanisms to leverage the advantages each sensor and achieve accurate detection [2].

1.2 Problem Statement and Research Aims

In this paper, we researched ways to design a pipeline that will allow us to leverage the advantages of multiple sensors and use them as one for the detection of target objects. Our initial goal was to implement a method called early fusion; therefore, we started by collecting and preparing a synchronized multi-sensor dataset that is specifically tailored to an early fusion pipeline. After a thorough examination of the raw, unannotated dataset, due to the practical challenges of fusing raw data, we have found early fusion to be infeasible for our project. Therefore the clear option was to go through with a late fusion pipeline. Although late fusion pipelines don't strictly require synchronized data, our dataset still provides synchronous data for possible future work [3]. Then the dataset was annotated and was used to train individual single-modality models. The performance of the individual models were later compared to performance of the complete pipeline.

1.3 Research Questions and Objectives

The main research question of this paper is to find out if the fused multi-modal approach of drone detection can be more accurate than the usage of single-modality detection models, and our objectives align with this question in the sense that our main objective is try and develop a full pipeline that implements such an approach and compare its performance against the baseline models.

2. RELATED WORK

The paper by Vajravelu et Al.

3. DATASET PREPARATION

3.1. Naming Convention

3.1. Extracting Individual Frames

3.3. Categorizing Frames

Guided by similar projects [4] we used electro-optical and the infrared sensors of a DJI Matrice 4T, because it provides us with the platform for hardware time-synchronized footage so that we can record scenes that contain drones, both in Electro Optical (EO) and Infrared (IR) views simultaneously [5]. The recorded footage was split into individual frames using scripts with the rate of 2 frames per second (fps), but this approach introduced a problem where the 0.03fps difference between the sensors caused our script to miss 1 frame every 33 seconds, shown by the formula:

$$\frac{1}{0.03} \approx 33.3$$

Figure 1 The difference in framerate causing a missing frame

This issue required us to move from a frame-number based approach to a time-based approach. We came up with a solution what we call “Slave-master Time based Slicing”. This method treats both sensors asymmetrically. Sensor 1 acts as the “master” and is read sequentially frame by frame just like how we would normally do. Then depending on the interval of the frames to be extracted (here its 2fps) it selects the frames to be extracted using sensor 1 feed along with timestamps of the individual frames. Then the method forces the sensor 2 (IR) feed to jump the timestamp that was decided in sensor 1 (EO) and extract that frame. Because the slave stream is anchored to the master stream clock rather than frame count, the same issue with frame-based extraction cannot occur. One important thing to mention is that the script assumes that both streams have the same initial time of 0 at the start.

At the end of this process, we had the initial version of the dataset which is described in Table 1. One more thing to note, which will be important later on is that the resolution of the images captured by different sensors, and their fields of view (FOVs) are different [5].

Table 1 Dataset Characteristics

Sensor	Resolution	Frame Rate	Total Samples
EO	4K / 1080p	29.97 fps	#getexact(paired)
IR	640x512	30 fps	#getexact

Following the creation of the dataset, we realized that in order to utilize the automation of tasks to any extent and ensure data retrieval, a proper standardized file naming convention must be established. The naming convention we have decided on structures the file names into seven parameters, each parameter describing a piece of information explaining the scene in the image. The naming convention’s structure is defined in Table 2, and the template of that structure is presented in Figure 2 along with its example.

Table 2 Dataset naming convention parameters

I: Item	0 -> Bird
	1 -> Drone
	2 -> No object
L: Light	0 -> Bright/visible
	1 -> Low brightness/visibility (rain, fog, snow)
D: Distance	0 -> Near (< 50 meters)
	1 -> Far (> 50 meters)
B: Background	0 -> Clear
	1 -> Cluttered
S: Sensor	0 -> EO (Optical)
	1 -> IR (Thermal)
EXP: Unique Experiment Name	
FID: 6-digit zero-padded frame ID	

Template: Item_Light_Distance_Background_Sensor_EXP_frameFID.jpg
Example: 1_000_0_ERF1_frame000000.jpg

Figure 2 Template of the file naming convention and an example of a named file

The example above can then be interpreted by analyzing its name in the following order, as the image of a drone, in a bright/visible scene, relatively close to the camera, the background being clear, captured by the EO camera, during the experiment ERF1, and it is the first frame from this experiment.

Since the frames were extracted from a given video, they automatically inherit the name of the video that they were sliced from and were given a unique ID. Although this is the proper way to name individual frames, we have realized that not all frames in a specific video really belonged in that category.



Figure 3 A frame that was categorized as cluttered, but should be categorized as clear

To address this issue and improve the quality of our datapoints, we have created a tool specifically built for this purpose, named BitFlipper, which allows us to quickly traverse images and rename them efficiently through an UI shown in Figure 4.

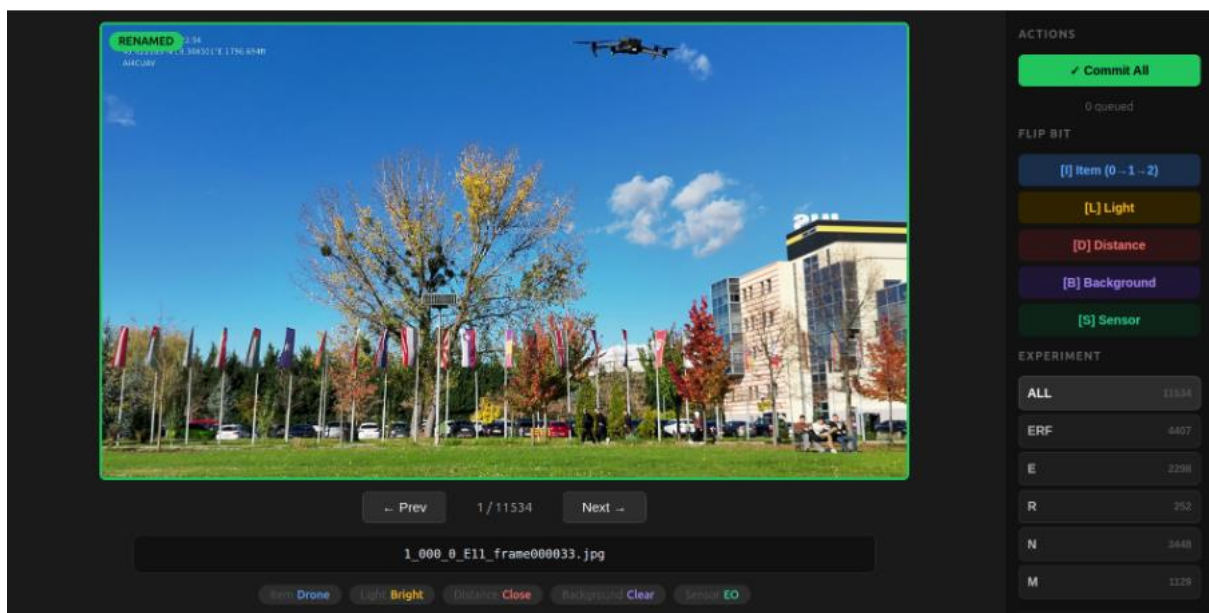


Figure 4 BitFlipper UI

To make sure the individual members have the same understanding of what a “cluttered” or “clear” scene is during renaming, we have come up with a decision graph. This way, we ensure consistency when renaming data points.

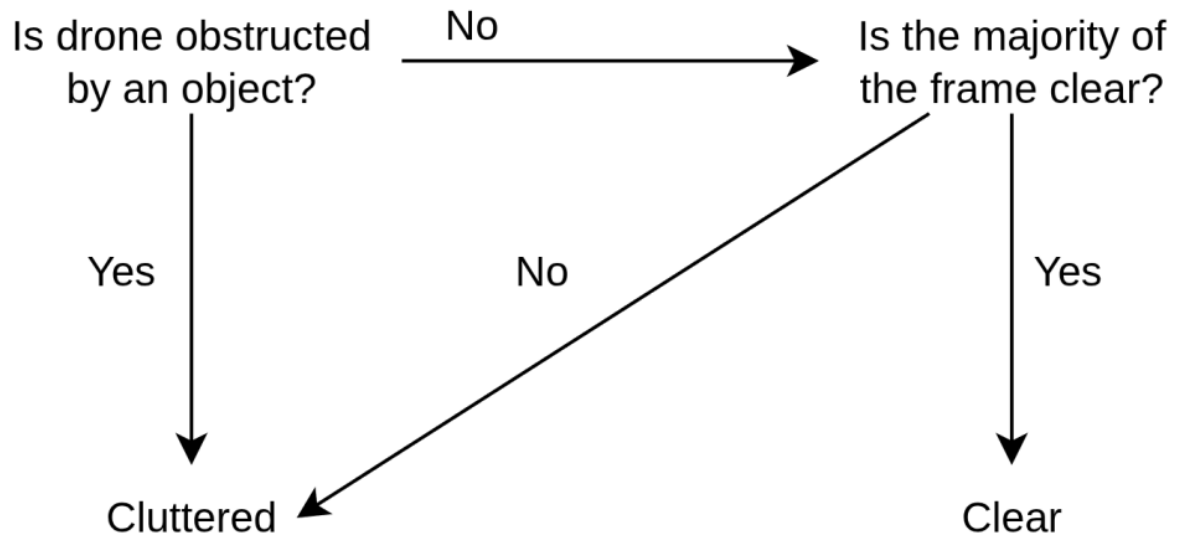


Figure 5 Decision scheme for categorizing frames as cluttered or clear

After renaming data points manually, we were able to classify 3164 pictures to their correct category.##### retrieve the information and statistics about the contents of the dataset as is presented in Table 3.

Table 3 Dataset Statistics

Category	Class	Count	Percentage
Item	Bird	0	0.0%
	Drone	11327	98.2%
	No Drone	207	1.8%
Light	Bright	8086	70.1%
	Low	3448	29.9%
Distance	Close	4950	42.9%
	Far	6584	57.1%
Background	Clear	9410	81.6%
	Cluttered	2124	18.4%
Sensor	EO	5744	49.8%
	IR	5790	50.2%

Several conclusions can be drawn from the Table 3, and along with additional explanation, a better overview of the dataset can be understood. Even though we have tried to create drone

image pairs of corresponding EO and IR images, the count of EO and IR images is not equal. This is because the process of data cleaning has removed counterparts from several pairs, but this is not the problem since the EO and IR baseline models will be trained separately. There is a significant count of all classes of images which provides dataset diversity to combat overfitting [6]. And lastly one can notice that there are no images of Bird class in our dataset, even though the naming convention seems to support it, that is because initially we planned to create a model that accommodates for the detection of birds as well, but we have narrowed our focus to the implementation of model fusion using only the images of drone. Additionally, 207 images lacking a drone exist to provide drone-less scenes in order to test models when drones are not present in the image.

4. IMAGE LABELING

Since our system's purpose was the accurate detection of drones on an image we needed to “teach” the drone detection algorithms by example, and thus our approach of training the models was by using supervised learning [7], which is the approach of training an object detection model using labeled datasets, datasets where there is associated metadata for each image explaining the contents of that image [8].

For labeling, we have opted for the usage of CVAT (Computer Vision Annotation Tool) running in Docker running on a local machine. CVAT platform that allowed us to manually label drones on images [9] using a rectangular box as shown in Figure 6, and export all annotation in the YOLO (You only look once) format, which stores the labeling data as one text file per image explaining the bounding box as a set of coordinates.

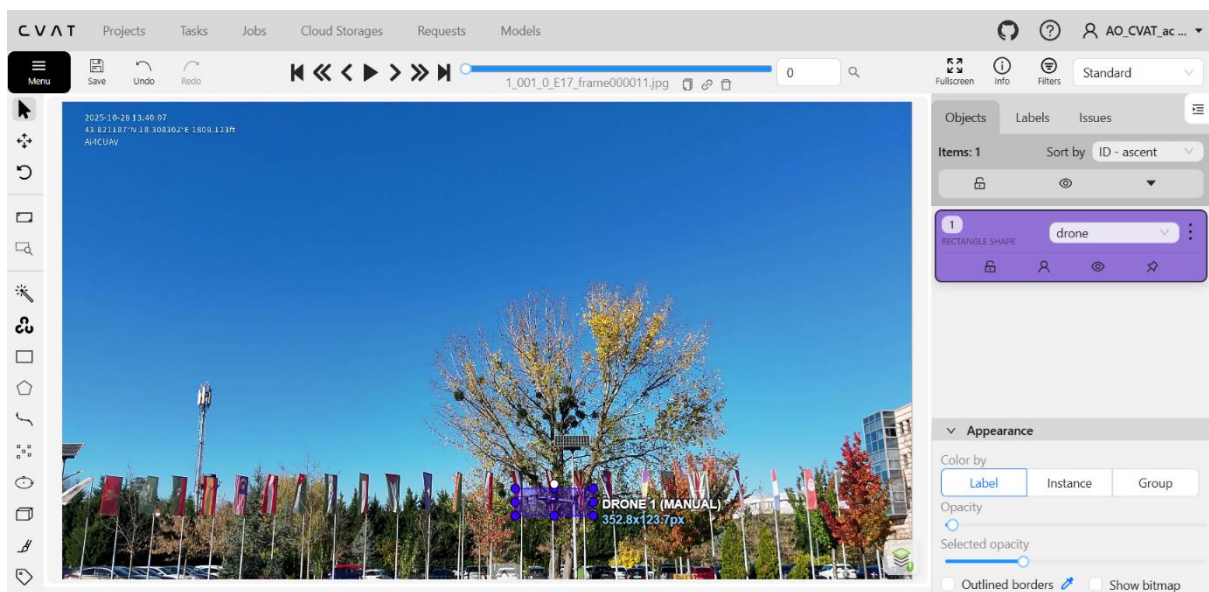


Figure 6 Image with a labeled drone on the CVAT annotation platform

The YOLO annotation format is structured as a text file as presented in Figure 7 in which a single label is presented as an array of 5 numbers, the first number presenting the class of the label (as in which type of object was labeled), the next two numbers present normalized X-Y coordinates of the center of the label on the image, and the last two numbers present the normalized width and height of the label [10].



Figure 7 A drone label in YOLO annotation format

5. BASELINE MODELS

The main goal of this project is to attempt creating a system that can combine the results of single modality models into a fused model where we hypothesize that the fused model could be the whole that is greater than the sum of its parts in terms of detection performance metrics, and the measure if such an attempt yields the hypothesized improvement in detection.

Thus, we needed to create the baseline models in order to be able to compare our approach to single-modality technique and measure the improvement present and to what degree.

Additionally, since we have opted for late fusion, something that will be expanded on in the following sections, we will be using the baseline models as modules of our fusion pipeline, thus the single-modality approaches will have two purposes, one serving as a baseline, and the other serving as modules of the fused system.

For the training of baseline models we have used the Ultralytics YOLO11 object detection model [11]. We have trained both models on 150 epochs, and for the EO model we have used only EO images, while for the IR model we have used only IR images. The performance metrics of both models will be discussed later in the Results section.

These models then work by taking an image as an input and return an object as an output that contains several fields, of which most interesting to us are the bounding box coordinates of the detected object and the confidence score that defines how confident the model is in its detection. The returned object values are presented in Figure 8 obtained from Ultralytics documentation [12].

Attribute	Type	Shape	Description
<code>result.bboxes</code>	<code>Boxes</code>	<code>(N)</code>	Detection boxes.
<code>result.bboxes.data</code>	<code>torch.float32</code>	<code>(N,6/7)</code>	Raw <code>[x1,y1,x2,y2,conf,cls]</code> , plus optional track ID.
<code>result.bboxes.xyxy</code>	<code>torch.float32</code>	<code>(N,4)</code>	<code>xyxy</code> pixel boxes.
<code>result.bboxes.conf</code>	<code>torch.float32</code>	<code>(N,)</code>	Confidence scores.
<code>result.bboxes.cls</code>	<code>torch.float32</code>	<code>(N,)</code>	Class IDs; cast to <code>int</code> for names.

Figure 8 YOLO detection object

6. FUSION

Multi-sensor data fusion is defined as the process of combining observations from multiple sensors in order to provide a scene analysis that is more complete and robust [13]. While there is a multitude of different approaches to implement fusion, the majority of them can be grouped into two styles of approaches, those being early fusion and late fusion [3].

Early fusion (also referred to as ‘data-level’, ‘image-level’, or ‘feature-level’) is focused around integrating data from various sensors into a more thorough representation, such as combining an EO image and an IR image into an image that contains both EO and IR data. A diagram of a detection system that implements early fusion is presented in Figure 9 obtained from [3].

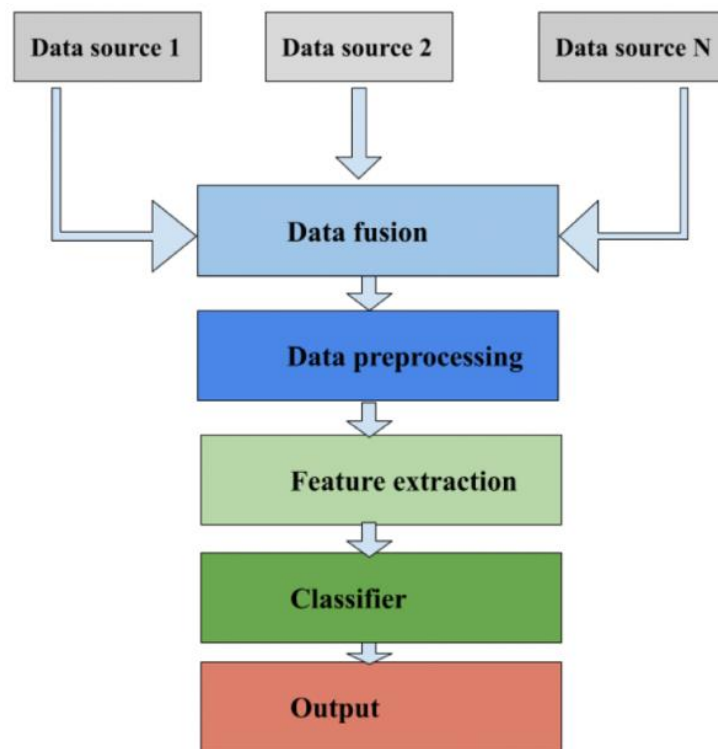


Figure 9 Detection system that implements early fusion

On the other hand, late fusion (also referred to as ‘decision-level’ fusion) is based around the implementation of different detection models analyzing their respective single-sensor inputs to provide their decisions and confidence scores, which will then be aggregated into a combined decision that takes account of all single-sensor models and combines them in a meaningful way. An example of late fusion being implemented is shown in Figure 10 obtained from [3].

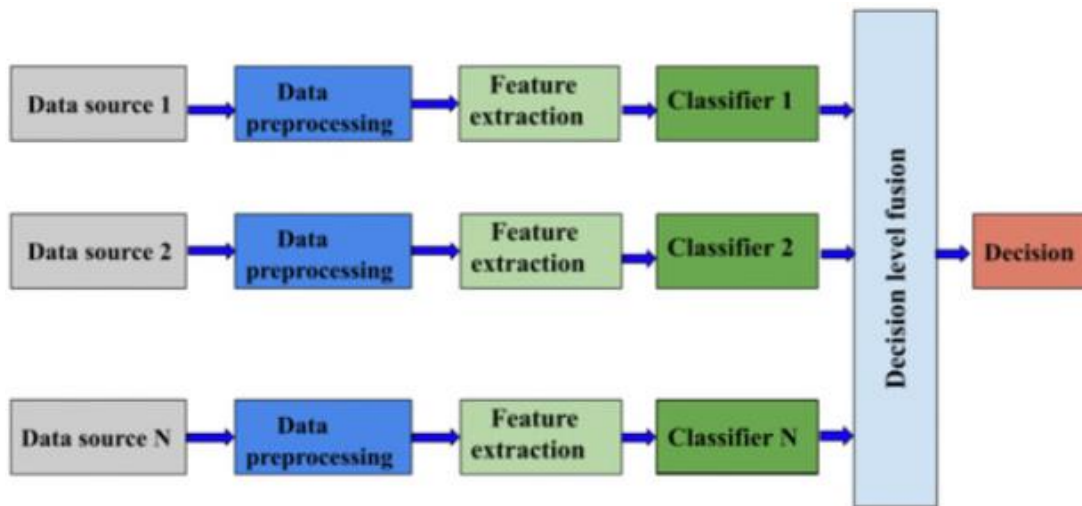


Figure 10 System that implements late fusion

While both early fusion and late fusion have their advantages [14], we primarily concerned ourselves with the convenience and the feasibility of implementing them. Early fusion requires proper temporal and pixel perfect spatial alignment of data from different sensors [15], and due to the IR and EO sensors having different FOVs and spatial resolutions this process would prove itself challenging [16], not the mention the difficulties that would arise from the IR and EO sensors capturing different details (light and heat signatures respectively) and not having a perfect temporal alignment. However, it is important to mention that late fusion does not remove the need for alignment entirely. Because the fusion engine associates EO and IR detections by bounding box overlap and to do that, these boxes need to be in the same coordinate space. We decided that by calculating a affine matrixwe could align the detection boxes. This is entirely different than warping and aligning every pixel on two sensors.

For this reason we have chosen the approach of using late fusion as the model of fusion we will attempt to implement, which will not only be more feasible in terms of implementation, but also quicker as the baseline models we have created for the purposes of result comparison can also be used as single-sensor models whose results will be aggregated in the process of fusion.

7. FUSION PIPELINE

A pipeline is defined as an architecture that models the system into a set of components that are connected through pipes so that data computations in a system are processed in a stream-like manner from a component to a component [17]. We deemed this architecture useful for our project since we have identified multiple atomic components that will process data in the mentioned stream-like manner.

An abstract overview of the system is presented in Figure 11, where there are multiple single-sensor models that output confidence scores and bounding boxes, and these outputs along with additional data that affect fusion are combined in a fusion engine to provide a final detection with confidence.

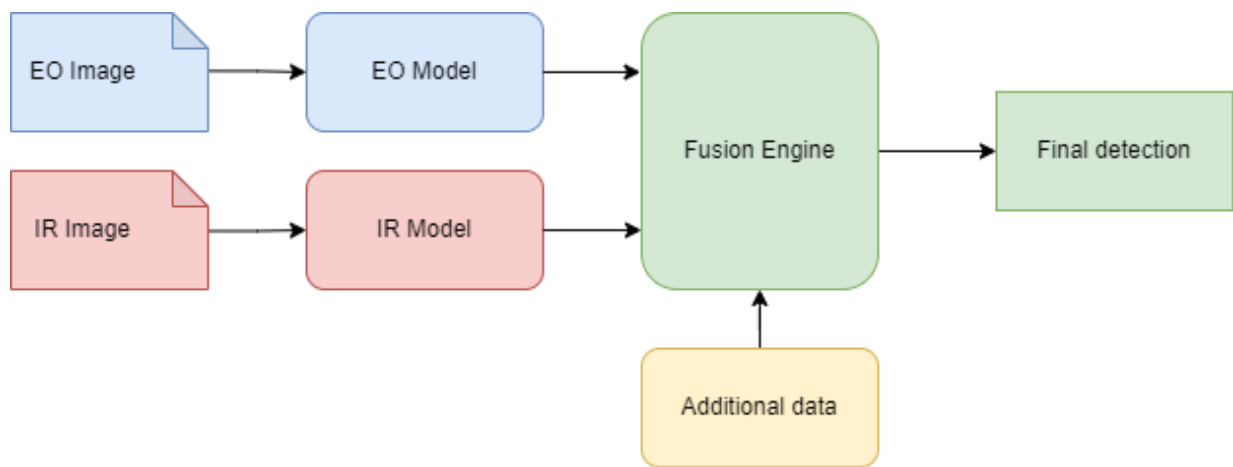


Figure 11 Abstract view of the system

The task then remains on deciding which additional data should affect fusion, how is the fusion designed, and what can we do ensure the system leverages fusion in order to acquire the hypothesized higher performance.

7.1 Input data selection

Aside from the EO model's and IR model's outputs, we must decide on which additional data should be used as inputs to the fusion engine that will affect the fusion results. This idea of using additional data stems from our plan of usage of fuzzy logic which will be explained in the following sections, where the main idea was that since different models have advantages in different scenarios we should rely on the model that is more appropriate to that scenario [4].

One idea was to use live data acquired from APIs (Application Programming Interfaces) that will return the current weather conditions of the place where the system would be implemented since weather conditions can affect image detection algorithms [18]. An example of such API is OpenWeather [19], that returns a comprehensive analysis of a location's weather including data on the current weather type and visibility.

Another idea is the calculation of the average brightness of the scene in order to obtain the information of whether a scene is dark or bright, since a common assumption is that EO models perform better in well-lit scenes, while IR models perform better in poorly lit scenes [4].

Our fusion engine conditions itself trusting two inputs, brightness value and target size. However, our framework is easily modifiable to enable the inclusion of additional inputs, including but not limited to weather condition data.

Concerning the analysis of brightness, our approach was to obtain the average brightness of the EO image since the EO image presents the optical description of the data (visible light) that gives us information on the brightness, unlike the IR image that only reads thermal data. Our method was convert the EO image from its standard RGB (Red Green Blue) to the HSV (Hue Saturation Value) format which gives us the Value (brightness) of each pixel, find the average value of the EO image, and then check in which of the brightness ranges does the average value fall in. We have decided to use Exponential Moving Average (EMA) on the brightness value. This ensures any sudden jitter or jump on the brightness value such as a car with headlights or a passing cloud doesn't cause the fusion weights to oscillate between frames. The EMA is calculated using the formula below.

$$EMA_i = ((Current\ Brightness \times a) + (EMA_{i-1} \times (1 - a)))$$

Equation 1 Exponential Moving Average equation

And then for our second condition we have target size. The target size is calculated from the long side of the predicted box. For an agreement pair, when two sensors are deciding on the target size, it's calculated from the mean of two long sides of the boxes.

The ranges are defined by the thresholds we have intuitively and empirically determined but can be modified easily for fine-tuning later.

Table 4 Brightness thresholds

Trust_IR	Small	Medium	Large
dark	0.90	0.85	0.80
dim	0.80	0.70	0.60
bright	0.60	0.45	0.35

7.2 Pipeline Stages

The fusion pipeline that we needed to implement needs to include the individual EO and IR models, the sensors that feed those models, the brightness analyzer, target size and the fusion engine/core that will combine all the available data and aggregate it into a final result that detects a drone within a scene.

The pipeline stages are presented in , and as we can observe, the pipeline begins with the collection of EO and IR footage of the scene. Since the fusion would be meaningless if the frames of EO and IR footage are of different moments, we need to make sure the timestamps

of both are within a certain temporal tolerance ($\pm \frac{1}{2}$ frame interval). Then the EO frame is processed in order to obtain the brightness of the scene as described in the previous section. This would yield in a result shown in Table 4, that will affect the fusion later on. (meanV = mean value; T_high = upper brightness threshold; T_low = lower brightness threshold)

Table 5 Brithness ranges

Regime	Condition	Meaning
DAY	meanV $\geq$ T_high	EO is more trustworthy
TWILIGHT	T_low $\leq$ meanV < T_high	EO and IR equally trustworthy
NIGHT	meanV < T_low	IR is more trustworthy

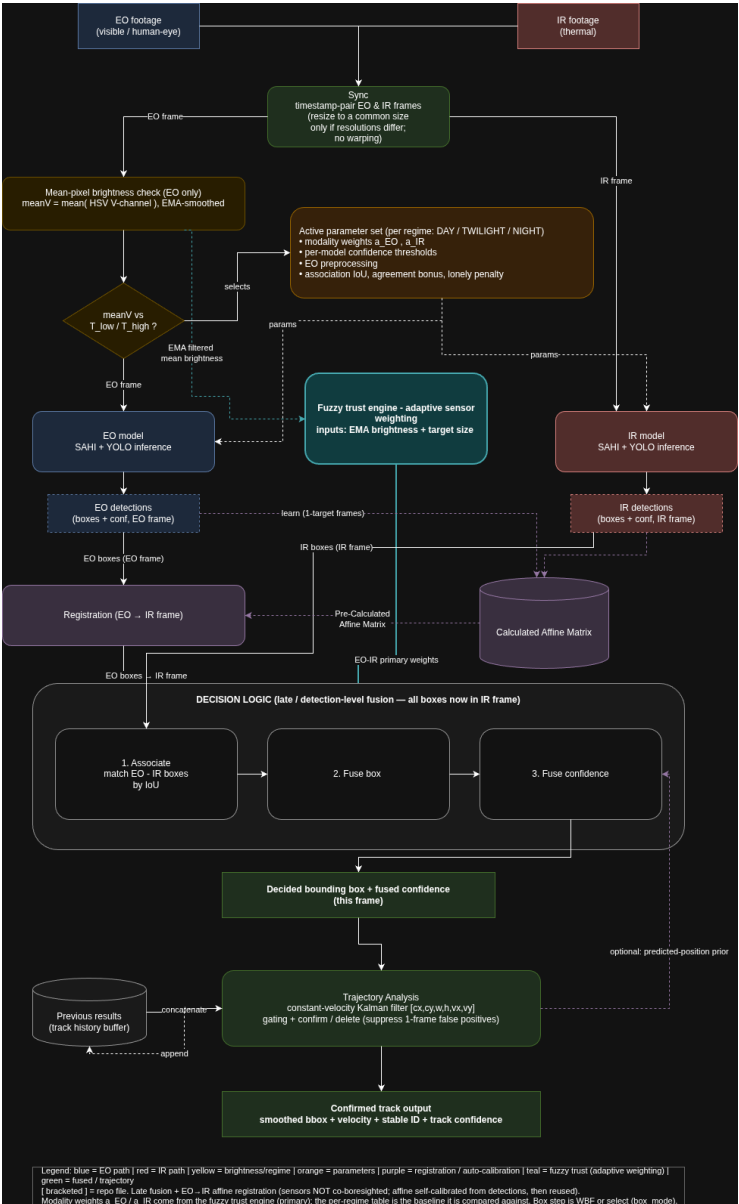


Figure 12 Fusion pipeline stages.

Following that, the individual EO and IR models are run on the EO and the IR frame respectively, and output the detection boxes and confidences as described earlier in Figure 8.

Because two sensors feed have different resolutions, EO prediction box will not be in the same coordinate system as IR. And since we calculate the result prediction box using the overlap of both predictions, they need to be on the same coordinate systems. To achieve this, we calculate a translation matrix called a affine matrix that maps one prediction box to another (median offset of 112px). This way we can compare both frames directly. Corresponding detection box centers are collected, outliers rejected with RANSAC and the fit is locked once it has enough inliers and a low median residual (~ 3.8 px on our data). At run time, every EO box is warped by the affine matrix into the IR frame and only after can we calculate IoU of both boxes.

We also made a proposal on the collection of previous fused results and using them for the purpose of trajectory analysis which can be used to analyze if the current detection result is likely an anomaly that can be attributed to an incorrect detection or not.

7.3 Decision Logic

The decision logic is the core of the late fusion, for which our approach is the adaptive weighting, which in our case is built on fuzzy logic. The adaptive weighting is the late fusion approach that takes the confidences from individual models, multiplies them by them by a weight assigned to them describing how reliable those models are in a given situation, and sums up all such products to yield a final certainty. Weighted Boxes Fusion works in a similar way, but focuses on the bounding boxes of the detected objects.

Fuzzy logic lets a value belong to several categories at once, each to a degree between 0 and 1, instead of a hard threshold. Our system takes each input (brightness and target size) and fuzzifies them into overlapping membership values. A small rule base maps every input combination to a trust value, and those are then combined by a weighted average (defuzzification process) into a single weight. The result is a trust value that changes smoothly as conditions change rather than jumping at a boundary.

```
w_eo = modality_weight_EO(regime) * conf_eo
w_ir = modality_weight_IR(regime) * conf_ir
box = (w_eo * box_eo + w_ir * box_ir) / (w_eo + w_ir)
```

The boxes of the EO and IR in the image above matched if they correspond to the same box, which is confirmed if for any pair of boxes (one of IR and one of EO) their IoU (Intersection over Union) **assoc_iou** is above a certain threshold (such as 0.4), otherwise the detected objects are not treated as the same object. If the objects correspond to the same object they will gain an **agreement bonus**, otherwise the single objects will get the single-sensor **lonely penalty** that will take affect in the fuzzy logic calculations.

The table of the fuzzy logic is presented in.

Parameter	DAY	TWILIGHT	NIGHT	Role
modality_weight_EO	0.70	0.50	0.30	Trust in EO

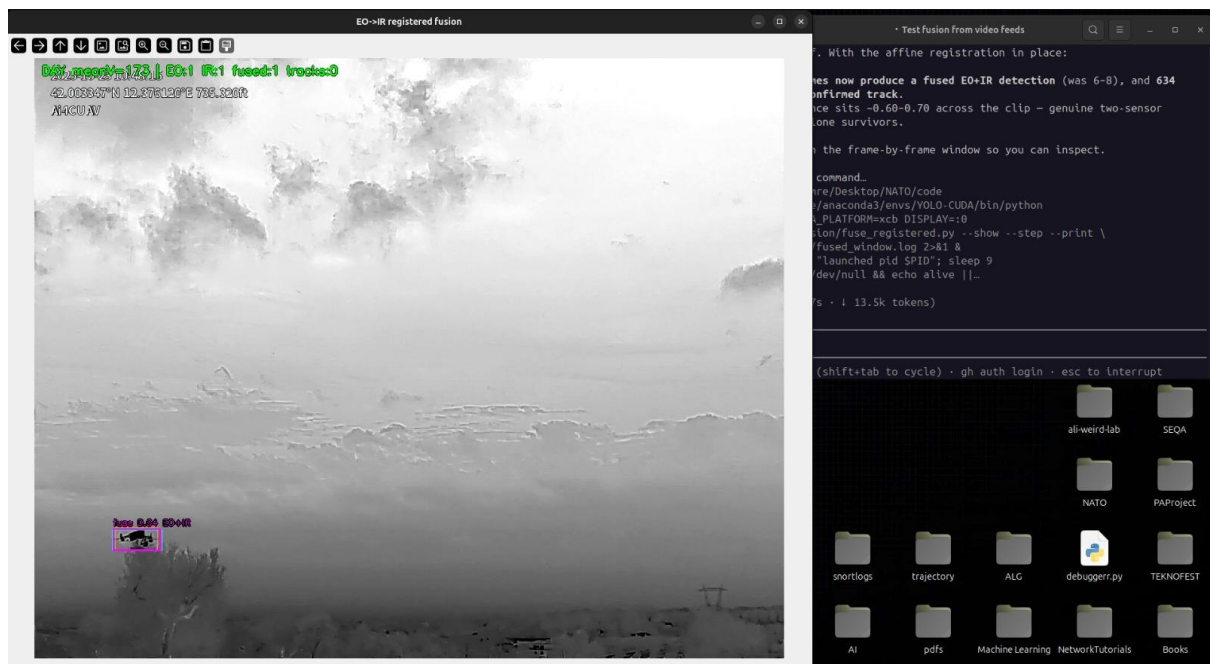
modality_weight_IR	0.30	0.50	0.70	Trust in IR
conf_thresh_EO	0.40	0.45	0.55	EO is noisier in dark
conf_thresh_IR	0.45	0.40	0.35	IR is stronger in dark
eo_preprocess	none	gamma	CLAHE+gamma	Help EO in low light
assoc_iou	0.40	0.40	0.40	EO/IR match threshold
agreement_bonus	0.15	0.20	0.20	
lonely_penalty	0.20	0.15	EO 0.40/IR 0.10	

8. RESULTS

We have provided several approaches to the detection of drones and compared their performances below (we will give the table a caption later on) (we will fill the table with the missing results as well)

Approach	Precision	Recall	F1-Score	MAP50-95	mAP50
EO baseline	0.87	0.5914	0.6899	0.54	0.64
IR baseline	0.79	0.68	0.73	0.51	0.73
EO 50%; IR 50%					
EO 70%; IR 30%					
EO 30%; IR 70%					
Adaptive weighting					

Below is the fused model in action successfully detecting a drone.



9. DISCUSSION

After the results table is filled in we can complete the discussion.

10. CONCLUSION

After the discussion is filled in, we can write the conclusion.

REFERENCES

- [1] A. Vajravelu, N. A. Kumar, S. Sarkar and S. Degadwala, Security Threats of Unmanned Aerial Vehicles, 2023.
- [2] M. A. Khan, H. Menouar, A. Eldeeb, A. Abu-Dayya and F. D. Salim, On the Detection of Unauthorized Drones - Techniques and Future Perspectives: A Review, 2022.
- [3] U. Sedaliyeva, L. Ilipbayeva, K. Taissariyeva, N. Smailov and E. T. Matson, Advances and Challenges in Drone Detection and Classification Techniques; A State-of-the-Art Review, 2023.
- [4] M. Felter, Decision-Level Fusion of Electro-Optic and Infrared Modalities for Improved Aerial Vehicle Detection, 2025.
- [5] "DJI Matrice 4 Specs," DJI enterprise, [Online]. Available: <https://enterprise.dji.com/matrice-4-series/specs>. [Accessed 2026].
- [6] J. P. Gygi, S. H. Kleinstein and L. Guan, "Predictive overfitting in immunological applications: Pitfalls and solutions," *Human Vaccines and Immunotherapeutics*, vol. 19, no. 2, 2023.
- [7] D. Valkenborg, M. Geubbelmans, A.-J. Rousseau and T. Burzykowski, "Supervised Learning," in *Statistic and Research Design*, AJO-DO, 2023, pp. 146-149.
- [8] Z.-H. Zhou, "A brief introduction to weakly supervised learning," *National Science Review*, vol. 5, no. 1, pp. 44-53, 2018.
- [9] O. Avrunin, Y. Samokhin, K. Gromaszek, V. Dudenko, I. Starkova, A. Arypzhon, O. Tiutiunnyk, A. Vitiuk and L. Jinqiong, "Optimization of cell annotation process: combining manual and automatic labeling in biomedical data analysis optoelectronic systems," *Proceedings*, 2025.
- [10] G. Jocher and R. Munawar, "Ultralytics supported dataset formats," Ultralytics, 12 11 2023. [Online]. Available: <https://docs.ultralytics.com/datasets/detect#supported-dataset-formats>. [Accessed 10 6 2026].
- [11] G. Jocher and R. Munawar, "Ultralytics yolo11," Ultralytics, 30 9 2024. [Online]. Available: <https://docs.ultralytics.com/models/yolo11#performance-metrics>. [Accessed 10 6 2026].
- [12] G. Jocher and Q. Burhan, "Ultralytics Object Detection," Ultralytics, 12 11 2023. [Online]. Available:

https://docs.ultralytics.com/tasks/detect?utm_source=chatgpt.com#models.
[Accessed 10 6 2026].

- [13] H. Durrant-Whyte and T. C. Henderson, "Multisensor Data Fusion," in *Springer HAndbook of Robotics*, 2016, pp. 867-896.
- [14] L. M. Pereira, A. Salazar and L. Vergara, A Comparative Analysis of Early and Late Fusion for the Multimodal Two-Class Problem, IEEE, 2023.
- [15] D. J. Dwyer, M. I. Smith, J. L. Dale and J. P. Heather, Real-Time implementation of image alignment and fusion, 2004.
- [16] C. Dionne-Pierre, S. Foucher, J. Théau, J. Lemaître, P. Charbonneau, M. Brousseau and M. Varin, Fusion or Confusion? Assesing the impact of visible-thermal image fusion for automated wildlife detection, 2025.
- [17] E.-E. Doberkat, Pipelines: Modelling a software architecture through relations, 2003.
- [18] C. Hellert and P. Stutz, Selecting Adequate Aerial Perceptual Functions with Fuzzy Logic, 2016.
- [19] "Openweather Documentation," Openweather, [Online]. Available: https://openweathermap.org/api/one-call-4?collection=one_call_api. [Accessed 10 6 2026].